Projet Java

Introduction

Ce projet est un exercice dans lequel vous devez faire la démonstration que vous savez utiliser les éléments que nous avons vus ensemble. Naturellement votre programme devra être bien conçu, il ne suffit pas "qu'il marche".

Sur ce sujet les IA génératives semblent ne pas donner de solution globalement satisfaisantes. Vous pouvez vous aider de ce qu'elles vous proposent, y chercher des exemples, mais il vous faudra de toutes façons tout retravailler plus finement car l'ensemble doit être cohérent. Et, surtout, vous devez être capable de défendre chaque partie de votre code à l'oral, à la fois en détail et dans la structure générale.

La soutenance sera longue, et donnera lieu à toutes sortes de questions. Votre précision à l'oral sera déterminante pour votre évaluation.

Généralités

- Le projet est à faire en binôme. Les monômes ne seront pas acceptés sauf un ou deux cas exceptionnels. Un forum Moodle sera ouvert pour mettre en contact ceux qui cherchent un partenaire. Il n'y aura absolument pas de trinômes, et il est entendu que si des travaux se ressemblent trop nous prendrons les sanctions qui s'imposent.
- Vous déclarerez la constitution de votre binôme rapidement en choisissant un groupe **binôme_xy** dans l'activité d'inscription mise en place sur moodle. Cela nous permettra de nous assurer que le travail a bien commencé, et d'organiser la suite.
- La soutenance aura lieu durant la période des examens. Votre travail sera à rendre quelques jours auparavant. Nous vous donnerons les dates exactes lorsque les réservations seront confirmées.
- Attendez vous à ce que la note de soutenance soit individualisée. Chacun doit donc maîtriser l'ensemble du travail présenté, y compris si une partie a plutôt été développée par votre camarade.

Présentation du jeu



Figure 1 – Tutorial

"Baba Is You" est un jeu de réflexion qui se représente sur une grille bidimensionnelle dans laquelle un personnage se déplace pour atteindre un objectif. Il devra pour cela interagir avec les objets présents.

Sur la figure 1 vous êtes le petit lapin blanc (qui ressemble surtout à une patate), et il vous faut atteindre le drapeau. Sur votre route, trois rochers vous font à priori obstacle. Il vous suffira d'en pousser un suffisamment pour vous ouvrir le passage. Tout cela est très classique.

Ce qui l'est moins ce sont les quatre phrases dans les coins, qui forment des règles qui expriment que :

- BABA IS YOU baba est votre personnage
- FLAG IS WIN c'est en atteignant le drapeau que vous terminez ce niveau
- WALL IS STOP les murs sont des obstacles
- ROCK IS PUSH les rochers sont déplaçables

Les membres gauche de cette règle correspondent à des objets présents sur la grille, et les membres droits sont des propriétés. Remarquez en particulier que ces mots occupent une case de la grille, et de fait il font partie du jeu : ces entités sont elles aussi déplaçables (toujours). En conséquence de quoi les règles peuvent évoluer : si la continuité des blocs de la phrase WALL IS STOP venait à être rompue, alors la règle ne s'appliquerait plus, et les entités pourraient traverser les murs. Dans la figure 1 baba (que vous contrôlez car c'est you) n'a pas accès aux règles qui sont protégées par les murs, mais il le pourra dans d'autres niveaux. Vous pouvez vous familiariser avec le jeu sur ces démos :

- https://kbhgames.com/game/baba-is-you (limité à quelques niveaux)
- https://www.lexaloffle.com/bbs/ ?tid=142638 (pas très lisible : utilisez les flèches et X)

Nous nous limiterons à un petit nombre d'objets et nous ne vous demandons pas d'écrire des combinaisons de règles qui comporteraient le mot de liaison AND, ni de phrases complexes.

Cependant :

- une entité représentant un mot peut apparaître plusieurs fois : plusieurs règles peuvent donc la concerner. Vous pourrez établir une priorité entre propriétés pour arbitrer les résolutions.
- la lecture d'une règle peut se faire dans deux sens : de gauche à droite, et de haut en bas. Cela permet d'augmenter la jouabilité.

Méthodologie

Nous vous conseillons de vous faire une idée incrémentale des difficultés que comporte ce sujet.

Comprenons nous bien : il ne s'agit pas de faire 4 versions, mais de vous assurer mentalement que votre modélisation intègre certains aspect avant de la retravailler pour en intégrer d'autres.

Voici un exemple de décomposition :

- En premier lieu : comprendre ce qu'il vous faut pour réaliser un jeu où l'on pousse des cubes. Dans cette phase vous réfléchirez à comment représenter un board très simplement, et comment en rendre compte visuellement. Faites vous une idée, mais n'allez pas au bout car d'autres difficultés vous attendent.
- La propriété d'être pushable ne concernera pas toutes les entités au même moment. Il y a aura des objets bloquants, des objets qui se feront traverser (donc des superpositions), et d'autres qui s'entredétruiront. A ce stade considérez les règles comme étant fixe : vous êtes toujours baba, les rochers sont toujours poussable, l'eau est toujours un élément de décors (superposable), les murs sont bloquants. Vous commencerez donc à décider de comment modéliser le fait qu'un objet peut potentiellement avoir une ou plusieurs propriété. Vous réfléchirez aussi à l'action principale du jeu : pousser en chaîne. Trouvez votre propre solution pour cette opération, mais assurez vous en particulier que vous pourriez ainsi contrôler plusieurs YOU de manière cohérente.
- Introduisez un système de règle, définissable dans le code dans une variable. Et appliquez le à chaque tour pour vous assurer que vous pourrez bien dynamiquement faire ensuite évoluer les propriété des entités. Testez par exemple en faisant en sorte qu'un tour sur deux une entité est push puis sera block au tour suivant, ou que vous pouvez changer qui est YOU . Déterminez les actions pour d'autres mots que block et push.
- Ajoutez un moyen de repérer les motifs des règles à partir d'une lecture de l'état du board. Vous vous approcherez alors d'une version finale.

Quelques indications supplémentaires

- La fonctionnalité `undo / redo` est une exigence naturelle pour permettre une bonne jouabilité. Vous pouvez vous y interesser.
- Travaillez votre modélisation pour que les manipulations des propriétés, et des objets qui les concernent, soient confortables, naturelles.
- Séparez bien le problème de la visualisation de celui de l'évolution du modèle de jeu.

Si vous avez le temps

Voici une liste d'idée dans laquelle vous pouvez piocher pour individualiser un peu plus votre travail.

- introduire PULL qui serait déclenché par une combinaison de CTRL et d'une flèche de direction.
- ajouter KEY et DOOR
- gérer les règles de la forme WALL IS BABA
- soigner l'environnement du jeu (sauvegarde, suite de niveaux, éditeur)
- jouer sur le contrôle en permettant des règles comme A IS LEFT etc ... (c'est à dire reprendre en partie l'idée de : https://miknugget.itch.io/out-of-ctrl)
- réfléchir au AND

Conseils habituels

- Sauvegardez régulièrement votre travail. Chaque fois que vous envisagez une modification importante, conservez bien la version antérieure afin d'éviter des catastrophes.
- Si vous avez quelque part dans votre code un bloc qui fait plus d'une trentaine de lignes, alors il est quasi-certain que vous devriez vous relire pour introduire une phase intermédiaire.
- Séparer le modèle et sa vue, c'est-à-dire ne mélangez pas les parties du code qui correspondent à ce qui est propre au jeu et celles qui correspondent à l'affichage.
- Illustrez ce que nous avons vu ensemble, il y a certainement des List, des **enum**, des héritages naturels (restez simple) etc ...

Aspects pratiques de l'évaluation

Rendu

Les travaux seront à déposer sur `Moodle` sous la forme d'une archive **binomeNumX.tar** (Utilisez impérativement ce format) [1] . Elle s'extraira dans un répertoire *`nom1-nom2`*, et contiendra :

- les sources et ressources (images ...) de votre programme ;
- un `README` qui indique succinctement ce qu'est ce projet, et comment effectuer la prise en main (compilation, exécution et utilisation). Le correcteur ne doit avoir que deux choses à faire [2] :
  1. décompresser votre dépôt dans une console unix ;
  2. lire votre fichier README et y trouver la ligne de commande qui lance le programme sur une console !
- un rapport au format *PDF* **rédigé** explicitant :
  - ce qui a été traité,
  - les aspects les plus significatifs pour comprendre votre code,

1. `tar cvf binomeNum18.tar fichiers...` crée l'archive mon_fichier.tar
2. Testez vous même votre rendu sur une autre machine

- les quelques problèmes connus,
- l'état des extensions que vous auriez commencé mais pas eu le temps d'implémenter.

  Ce rapport ne doit pas être verbeux, ne cachez pas l'essentiel par un volume de baratin généré.
- toute chose utile qui pourraient rendre la soutenance fluide.

Soutenance

Elle se déroulera devant un ou deux enseignants dans un mélange de questions et de tests. Vous commencerez par une démonstration afin que nous ayons rapidement une vue d'ensemble ; nous étudierons votre architecture qu'il vous faut être capable d'expliquer clairement ; vous nous guiderez dans votre code lorsque nous chercherons à vérifier la cohérence entre vos explications et votre réalisation. Il pourra vous être demandé d'écrire quelques lignes de code. Votre style sera scruté.

Vous pouvez préparer des supports pour répondre rapidement à des questions que vous anticiperiez. Dans tous les cas, il faudra privilégier les échanges avec le jury pour nous convaincre de vos acquis.